COMP1511 26T3 · 第 3 周（W3）知识点整理

编写日期：2026-10-15 · 资料来源：仅限课程公开网站 https://cgi.cse.unsw.edu.au/~cs1511/26T3/（中文概括，非原文转载；技术术语与代码保留英文）

本周主题一览：讲座 —— struct 内含 enum（Pokemon 例子）→ **functions**（prototype、definition、call、`void`、参数与返回值）→ scope、local / global variables、pass by value → **Style**（Style Guide、`1511 style`）→ 简写 `++` / `+=` 与 `for` 循环 → **arrays**（hurdle 考点）与把数组传入函数。
Tutorial 与 Lab 本周主要练习上周讲的 while / 嵌套 while / scanf 循环 / struct + enum。
行政提醒（讲座公告）：Lab 3 截止 W4 **周二** 18:00（W4 周一为公众假期）；Lab 4 截止 W5 周二 18:00；W4 有 census date；W4 开始 Revision Sessions；Assignment 1 于 W4 初发布。

1. 复习：struct 里放 enum

```
enum element {FIRE, WATER, GRASS};

struct monster {
    enum element kind;
    int hp;
    double weight;
};

struct monster m;
m.kind = WATER;
m.hp = 60;
```

讲座用 Pokemon 例子把"一组相关数据（struct）+ 一组固定取值（enum）"组合起来，然后把重复的打印代码改写成函数。

2. Functions（函数）

2.1 为什么要写函数

- **Modularity**：把大程序拆成职责清晰、名字有意义的小块，便于阅读，也能单独测试和调试。
- **Reusability**：需要同样逻辑时调用函数，而不是复制粘贴。
- **Abstraction**：调用者只关心"做什么"，不必关心"怎么做"。
- 其实我们一直在用函数：`main` 本身，以及 `stdio.h` 中的 `printf`、`scanf`。

2.2 设计 → prototype → definition → call

写代码前先想清楚：函数叫什么？需要哪些输入（参数及类型）？返回什么类型？然后写出 **function prototype**：`返回类型 函数名(参数列表);`

```c
#include <stdio.h>

#define FEE_PER_DAY 3

int library_fee(int days_late);        // prototype: 写在 main 之前
void print_banner(void);               // 无返回值、无参数

int main(void) {
    print_banner();
    int fee = library_fee(4);          // 调用: 传入 argument, 接收返回值
    printf("Fee: $%d\n", fee);
    return 0;
}

// 根据逾期天数计算罚款
int library_fee(int days_late) {       // definition
    int fee = days_late * FEE_PER_DAY;
    return fee;                        // return 结束函数并带回一个值
}

// 打印欢迎横幅
void print_banner(void) {
    printf("=== City Library ===\n");
}
```

- 调用时 argument 的**个数、顺序、类型**都要和参数列表一致。
- 调用时当前代码暂停 → 执行函数体 → 遇到 `return` 或函数末尾时返回 → 从调用处之后继续执行。
- 函数最多返回**一个**值；返回类型为 `void` 表示不返回值（也叫 procedure），仍可用 `return;` 提前结束。`void` 函数不能用在需要值的地方（如 `int x = f();`）。
- 参数列表写 `void` 表示不需要参数。
- 有返回值的函数可以直接用在条件里：`if (library_fee(d) > 10)`、`while (scanf("%d", &n) == 1)`。
- struct 和 enum 也可以作为参数类型和返回类型。

2.3 函数与 Style

- 好的布局：`main` 放在文件上部，其它函数写在下面；因此要在 `main` 前写 prototype（编译器自上而下处理代码，调用前必须先见到声明）。
- 每个函数的实现前都要有注释，说明用途及副作用；函数名要有意义。

3. Scope、局部变量与 pass by value

- 代码块以 `{ }` 划分。变量的 **scope**：从声明处到所在块结束。
- 函数内声明的是 **local variable**：函数被调用时创建、返回时销毁，函数外访问不到。
- 在所有函数外声明的是 **global variable**——**COMP1511 禁止使用**。
- `int`、`char`、`double`、`enum`、`struct` 都是 **pass by value**：函数拿到的是一份**拷贝**，在函数里修改不会影响调用者的变量。

```
void reset(int score) {
    score = 0;                  // 只改了自己的拷贝
}

int main(void) {
    int score = 95;
    reset(score);
    printf("%d\n", score);      // 仍然是 95
    return 0;
}
```

若想让函数"改变"结果，目前的做法是**返回新值**并在调用处赋值：`score = new_score(score);`（更多方法见 W5 pointers）。

4. Style（代码风格）

- 代码是写给人看的：易读 → 错误更少、开发更快。**作业有专门的 style 分数**，期末考试代码也由人工批改。
- 课程有 Style Guide；可用 `1511 style 文件.c` 自动检查（修改风格后记得确认仍能编译）。
- 边写边整理的要点：
  - 必要的地方写注释；变量名反映用途；
  - `{}` 内缩进 4 个空格，右括号与开启该块的语句垂直对齐；
  - 一行只写一个表达式；空格使用保持一致；
  - 每行不超过 80 个字符；
  - 避免过深的 if 嵌套；
  - 把代码拆成函数。

5. 简写与 for 循环

完整写法	简写
`n = n + 1;`	`n++;`
`n = n - 1;`	`n--;`
`n = n + 5;`	`n += 5;`
`n = n - 5;`	`n -= 5;`

`for (初始化; 条件; 更新) { ... }`：初始化在循环开始前执行一次；条件在每轮开始前检查，为假则退出；更新在每轮结束时执行。它只是 counting while loop 的简写，特别适合已知次数的循环。

```c
// 两段代码效果相同：打印 0..4
int k = 0;
while (k < 5) {
    printf("%d\n", k);
    k++;
}

for (int k = 0; k < 5; k++) {
    printf("%d\n", k);
}
```

6. Arrays（数组，hurdle 考点）

6.1 概念

- 存放**多个同类型**值的集合（homogeneous，区别于可混合类型的 struct）。
- **Contiguous**：元素在内存中连续存放；**random access**：可直接访问任一元素；**indexed**：整个集合只有一个名字，用下标访问元素。
- 本课程会学两种重要的数据结构：array（现在）和 linked list（flexibility week 之后）。

6.2 声明、初始化与访问

```c
#define DAYS 5

int steps[DAYS];                          // 5 个 int，值未初始化
int rain[DAYS] = {3, 0, 12, 7, 1};        // 声明时初始化
int counts[DAYS] = {};                    // 全部初始化为 0

rain[2] = 9;                              // 下标从 0 开始：rain[2] 是第 3 个元素
// rain[5] 越界！合法下标是 0..4，dcc 会报运行时错误
```

- `{...}` 这种初始化语法**只能在声明时**使用，不能事后整体赋值。
- 不能直接 `printf` / `scanf` 整个数组，也不能用 `=` 把一个数组整体赋给另一个；都要**逐个元素**处理。

6.3 遍历（traversal）

```c
int total = 0;
int max = rain[0];
for (int i = 0; i < DAYS; i++) {
    total += rain[i];             // 求和
    if (rain[i] > max) {          // 找最大值
        max = rain[i];
    }
}
for (int i = 0; i < DAYS; i++) {  // 每个元素加 1
    rain[i]++;
}
```

讲座 `game_scores.c` 演示了：读入分数、打印（while 与 for 两种写法）、求和、整体加分、筛选打印、找最大值。

6.4 数组与函数

- 函数需要单独接收数组的**大小**：`void show(int size, int data[]);` 这样可以接受任意长度的数组。
- 传入函数的**不是**数组的拷贝：函数可以直接读取并**修改**调用者的原数组（因此可以用函数来填充数组）。
- 不能从函数返回一个数组（看起来能写，但结果会出错，课程后面会解释原因）。

```
void read_values(int size, int data[]) {
    for (int i = 0; i < size; i++) {
        scanf("%d", &data[i]);
    }
}

void show(int size, int data[]) {
    for (int i = 0; i < size; i++) {
        printf("%d ", data[i]);
    }
    printf("\n");
}
```

7. 常见坑（Common pitfalls）

坑	说明 / 正确做法
函数在定义之前被调用、又没写 prototype	编译报错/警告；在 main 前写 prototype
argument 顺序或类型与参数不符	结果错误或编译错误
有返回值的函数忘写 `return`，或调用后忽略返回值	结果丢失
以为函数内修改参数会影响 main 中的变量	pass by value 只改拷贝
使用 global variable	课程禁止，风格扣分
数组下标从 1 开始或循环写成 `i <= SIZE`	越界访问（off-by-one）
声明后再用 `arr = {...}` 赋值、或 `arr2 = arr1`	不能编译，需逐个元素赋值
数组没初始化就累加	垃圾值
嵌套循环中内层计数器没在每行重置	（tutorial 手动执行题的典型错误）
while 条件里的变量未初始化、或条件永远不会变假	未定义行为 / 死循环（tutorial 的手动执行题专门设置了这类陷阱）
缩进混乱、行过长、magic number	style 扣分；用 `1511 style` 检查

8. 本周 Tutorial（Week 03）

1. **While loops**：复习 count / sentinel / conditional 三种循环；把 flowchart 转成 C 代码；分组手动执行（hand execute）几段 while 循环并写出输出——其中有些故意包含未初始化变量、条件永不改变、条件一开始就为假等情况。
2. **2D while loops**：分组把几种 4×4 的 O/X 图案与对应的嵌套循环代码配对（对角线、隔列、边框等条件）。
3. **Scanning and loops**：判断以下场景应使用哪种循环并写出条件——遇负数停止求和；输入 `q` 前统计字符数；读到输入结束并打印偶数；累加直到达到用户给定目标。
4. **Structs and enums**：Coffee Shop 程序——定义饮品种类的 enum 和存放种类、糖数、杯型的 struct，读入订单并按规则计算价格（基础价，加大杯、特定饮品、每份糖各有附加费）。

9. 本周 Lab（Lab 03）

截止：**W4 周二 18:00**（因 W4 周一公众假期）。先 `1511 autotest` 自测，再 give 提交。部分题禁止使用数组（须用循环直接输出）。

难度	题目	内容概括	提交命令
☆	Check Off	课内与 tutor 完成	—
●◌◌	Count Up/Down	读入整数 n，逐行打印 0 到 n（n 为负数时倒数）	`give cs1511 lab03_count_up_down count_up_down.c`
●◌◌	Debugging - Factorial	修复一个计算阶乘的程序	`give cs1511 lab03_debug_factorial debug_factorial.c`
●◌◌	Print a Coordinate Grid（pair）	读入 n，打印 n×n 网格并显示每格坐标；不能用数组	`give cs1511 lab03_print_grid print_grid.c`
●●◌	Vector Sum	补全程序：读入两个 3 分量整数向量（struct），求和后存入新向量并打印	`give cs1511 lab03_vector_sum vector_sum.c`
●●◌	Custom Cake	先读层数，再为每层读一个字符，按要求打印蛋糕图案（先运行示例程序体会 char 的读入方式）	`give cs1511 lab03_custom_cake custom_cake.c`
●●◌	Letters Between（pair）	读入起点字母与目标字母，计算沿字母表向前或向后走的最短路径	`give cs1511 lab03_letters_between letters_between.c`
●●●	Draw a fancy pattern	读入奇数 n，用 `*` 和 `-` 打印指定的 n×n 图案	`give cs1511 lab03_xtreme xtreme.c`
●●●	Perfect Number	持续读数直到输入结束，判断每个数是否为完全数（等于除自身外所有因数之和）	`give cs1511 lab03_perfect_number perfect_number.c`
☠	Spiral / Decimal Spiral（extra-hard）	不用数组打印 n×n 螺旋图案（星号版 / 数字版）	（详见题面）
不计分	Debugging - Square；SplashKit	调试练习；SplashKit 图形界面入门活动	—

9.1 Extra Revision 与 Challenges（可选）

- Revision 03：Simple Counting Up、Simple Counting Down、Print a Solid Square、Print an Outlined Rectangle、Print a Z。
- Challenges 03：Sawtooth（●●◌）、Circle Generator（●●◌）、Numbered Hourglass（●●●）。

10. 本周讲座示例代码（Lecture Code）

目录	文件	演示内容
0.recap_loops	pattern.c	嵌套循环打印数字图案
1.recap_custom_data_types	struct_student.c、enum_weekdays.c、pokemon*.c	struct 内含 enum
2.functions	add_numbers.c、cat_cafe/（cat_cafe.c、cat_sound.c、welcome.c）、pokemon_functions.c / pokemon_no_functions.c、scope.c、memory_scope.c、pass_by_value.c、scanf_loop.c	函数设计与调用、void、scope、pass by value、用函数消除重复代码
3.style	bad_style.c、extra_style_examples/（bad_style、good_style、good_style_functions）	风格好坏对比
4.arrays	simple_array.c、game_scores.c、game_scores_functions.c	数组基本操作与传入函数

11. Check-off 自测题 推测

> 官方 check-off 页面（/check-off/03/）需 zID 登录，**具体题目「未发布」（对公众不可见）**。以下为依据本周内容**推测**的自测题，只列问题、不给答案。

1. 【推测】function prototype 包含哪三部分？为什么要写在 main 之前？
2. 【推测】`void` 作为返回类型和作为参数列表分别是什么意思？
3. 【推测】函数里修改一个 int 参数，main 中的变量会变吗？为什么？
4. 【推测】什么是 local variable？为什么本课程不允许 global variable？
5. 【推测】把一段 while counting loop 改写成 for loop。
6. 【推测】`int a[6];` 的合法下标范围是多少？访问 `a[6]` 会怎样？
7. 【推测】为什么把数组传给函数时还要传 size？函数能修改原数组吗？
8. 【推测】手动执行一段嵌套 while 代码，写出它打印的图案。
9. 【推测】给定一段风格很差的代码，指出至少三处 style 问题。
10. 【推测】如何用循环求数组的和与最大值？

来源页面（本章引用）

- 课程首页：https://cgi.cse.unsw.edu.au/~cs1511/26T3/
- Week 3 Lecture 1 幻灯片：https://cgi.cse.unsw.edu.au/~cs1511/26T3/slides/week_3/COMP1511_26T3_Week_3_Lecture_1_.pdf
- Week 3 Lecture 2 幻灯片：https://cgi.cse.unsw.edu.au/~cs1511/26T3/slides/week_3/COMP1511_26T3_Week_3_Lecture_2_.pdf
- Week 3 Lecture Code：https://cgi.cse.unsw.edu.au/~cs1511/26T3/code/week_3/
- Tutorial 03：https://cgi.cse.unsw.edu.au/~cs1511/26T3/tut/03/questions
- Lab 03：https://cgi.cse.unsw.edu.au/~cs1511/26T3/lab/03/questions
- Revision 03：https://cgi.cse.unsw.edu.au/~cs1511/26T3/revision/03/questions
- Challenges 03：https://cgi.cse.unsw.edu.au/~cs1511/26T3/challenges/03/questions
- Style Guide：https://cgi.cse.unsw.edu.au/~cs1511/26T3/resources/style_guide.html
- Check-off 03（需登录，未能访问）：https://cgi.cse.unsw.edu.au/~cs1511/26T3/check-off/03/

说明：本文件为中文概括整理，代码示例均为改写的简短示例；以课程官网为准。